
Práctica 6: Render pipeline y render features (2)

Objetivo: Practicar con URP

1. Instrucciones

La práctica se realiza sobre un proyecto de Unity en el que se van añadiendo escenas con distintos elementos y que utilizan shaders, materiales y renderers distintos.

Se entregará el proyecto completo de Unity, dejando únicamente los directorios necesarios (Assets, Packages y ProjectSettings). En el directorio Assets habrá, además, una carpeta Capturas con imágenes de pantalla que muestren el resultado de cada una de las tareas pedidas.

2. Punto de partida

[NO ES IGUAL al de la práctica anterior]

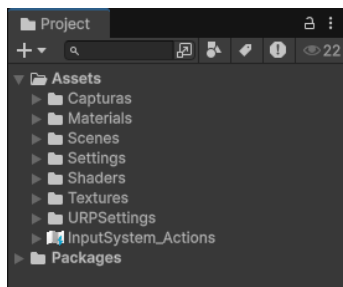
Crea un proyecto 3D en Unity que utilice el *Universal Render Pipeline*. Añade un directorio URPSettings para sus assets. Antes de empezar con la primera parte:

- Crea un asset “*URP Asset (with Universal Renderer)*” en la carpeta anterior llamado PR06_URenderPipelineAsset. Creará también un *renderer* que se utilizará por defecto PR06_URenderPipelineAsset_Renderer.
- Configura el proyecto (opción *Graphics* y quizá en las *Quality Settings*) para que utilice esa configuración.

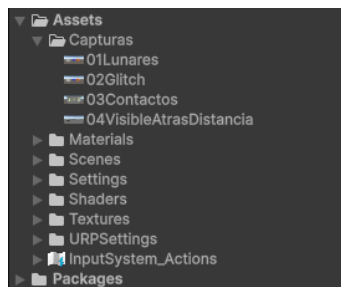
En la última parte de la práctica se utilizará ese renderer (será preferible *forzar su uso* en lugar de dejar el renderer por defecto, que cambiaremos para hacer pruebas cómodamente).

En la primera parte cada escena tendrá un renderer distinto, para lo que habrá que:

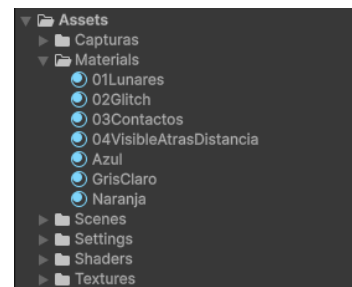
- Crear el nuevo renderer “*URP Universal Renderer*” con el nombre concreto indicado en la carpeta URPSettings.



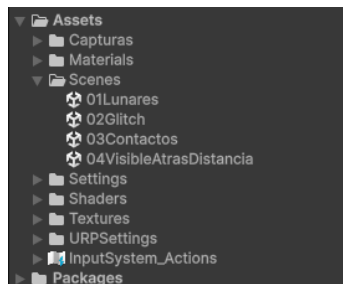
(a) Carpetas del directorio Assets



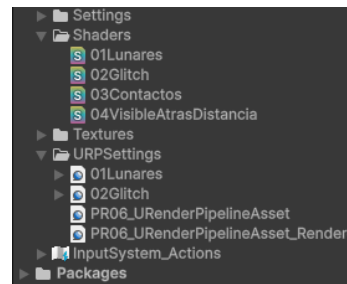
(b) Contenido del directorio Capturas



(c) Contenido del directorio Materials



(d) Carpetas del directorio Scenes



(e) Contenido del directorio Shaders y URPSets

Figura 1: Contenido del directorio Assets de la entrega

- Añadir el renderer en la lista de renderers de PR06_URenderPipelineAsset.
- Seleccionar ese renderer en la cámara de la escena.
- Aunque no es obligatorio, seguramente quieras poner el nuevo renderer como el renderer por defecto para que se utilice también en la pestaña *Scene* (si no has forzado el uso del renderer inicial en las primeras escenas, esta acción alterará su funcionamiento).

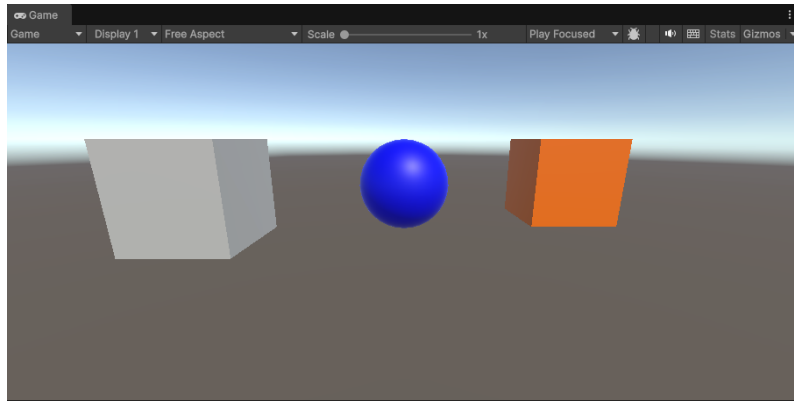
Crea en la carpeta Materials los siguientes materiales, basados en el shader por defecto de URP, Universal Render Pipeline/Lit:

- Azul que utilice el RGB (0, 0, 255).
- Naranja con (255, 120, 0).
- GrisClaro con (200, 200, 200).

Coloca en la escena los siguientes objetos:

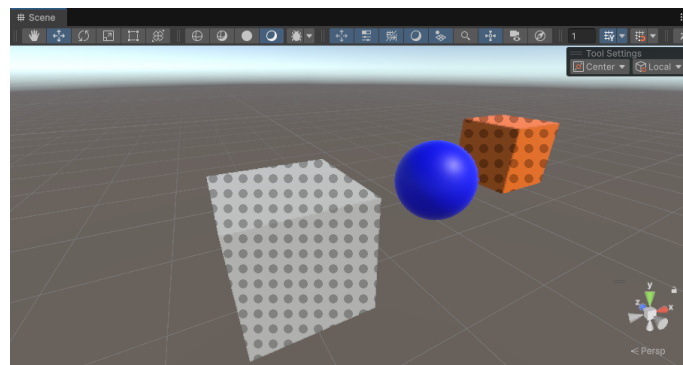
- Cubo GrisClaro en L: (-2, 0.5, 0).
- Esfera Azul en L: (0, 0.5, 0).
- Cubo Naranja en L: (2, 0.5, 0).

Configura la cámara de la escena en L: (0, 1, -3) y R: (15, 0, 0).



3. Lunares

El objetivo de esta práctica es hacer que todos los objetos que pertenezcan a la *layer* *EfectoEspecial* sean dibujados con unos “lunares”: círculos situados a modo de trama en la pantalla que oscurecen la zona ocupada por el objeto (multiplican su color por 0.5):



El efecto se basa en un shader *01Lunares* con los parámetros:

```
Properties
{
    [IntRange] _Dist("Distancia entre círculos", Range(0, 100)) = 20
    [IntRange] _Radio("Radio", Range(0, 50)) = 6
}
```

El shader lee el color de la textura con la información del frame tras el dibujado de opacos y lo oscurece dependiendo del valor de los parámetros (dados en píxeles).

Recuerda que la forma de convertir una posición en *screen space* a píxeles es:

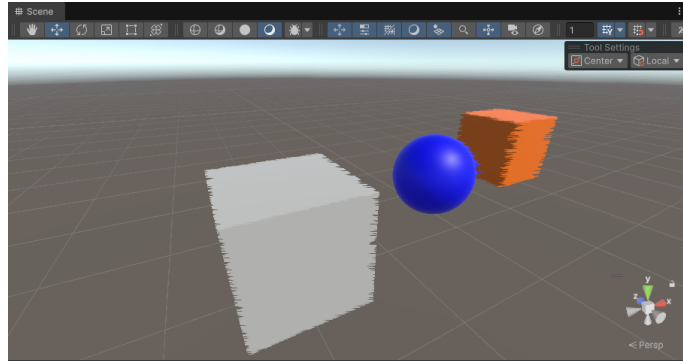
```
float2 screenUV = i.posSS / i.posSS.w;
float2 screenXY = screenUV * _ScreenParams.xy;
```

Tendrás que:

- Crear una nueva escena *01Lunares* con el contenido descrito en la sección 2.
- Crear la *User layer 3* llamada *EfectoEspecial* y asignarsela a algunos de los objetos.
- Crear un nuevo renderer *01Lunares* y añadir la *Render feature* necesaria.

4. Glitch

En esta ocasión debes añadir una perturbación en los objetos que pertenezcan a la etiqueta `EfectoEspecial` anterior:



La idea de funcionamiento es similar a la del efecto de los lunares pero ahora el shader tendrá un único parámetro:

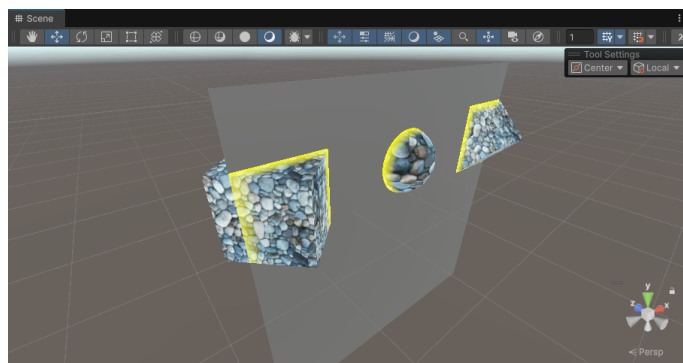
```
Properties
{
    [IntRange] _MaxDesplX("Maximo glitch horizontal (pixeles)", Range(0, 100)) = 10
}
```

Y utilizando la función de ruido ya utilizada con anterioridad y la parte entera del tiempo (`_Time.w - trunc(_Time.w)`), elegirá un desplazamiento horizontal para cada Y de la pantalla de forma que desplazará esa fila un número de píxeles entre 0 y `_MaxDesplX`.

5. Destacando los contactos

Abandonamos los Render Features para recuperar la creación de efectos completos en un único shader/material.

El objetivo es hacer un material que destaque el contacto con otros objetos con un color concreto (amarillo en la captura) cuando la distancia entre ambos entra dentro de un umbral.



Para probarlo, crea una escena nueva `03Contactos`, utiliza el renderer por defecto creado en la sección 2 (`PR06_URenderPipelineAsset_Renderer`) y añade un plano en L: (0, -1.8, -1), R: (-90, 0, 0), S: (0.5, 0.5, 0.5) con el color por defecto (`Lit`).

Crea un shader y material nuevo 03Contactos y parte del shader que simplemente muestra la textura:

```
Shader "Unlit/03Contactos"
{
    Properties
    {
        _MainTex ("Base (RGB)", 2D) = "" {}
    }
    SubShader
    {
        Tags {
            "RenderType" = "Opaque"
            "Queue" = "Geometry"
            "RenderPipeline" = "UniversalPipeline"
        }

        Pass
        {
            Tags { "LightMode" = "UniversalForward" }

            HLSLPROGRAM

            #pragma vertex vsMain
            #pragma fragment psMain

            #include "Packages/[...]/Core.hlsl"

            CBUFFER_START(UnityPerMaterial)
            sampler2D _MainTex;
            float4 _MainTex_ST;
            CBUFFER_END

            struct VsIn {
                float4 vertex : POSITION;
                float2 texcoord : TEXCOORD0;
            };

            struct VsOut {
                float4 pos : SV_POSITION;
                float2 uv : TEXCOORD0;
            };

            VsOut vsMain(VsIn v) {
                VsOut o;
                o.pos = TransformObjectToHClip(v.vertex.xyz);
                o.uv = TRANSFORM_TEX(v.texcoord, _MainTex);
                return o;
            }

            float4 psMain(VsOut i) : COLOR {
                return tex2D(_MainTex, i.uv);
            }
            ENDHLSL
        }
    }
}
```

```
}
```

El shader recibe dos parámetros adicionales:

Properties

```
{
    _MainTex ("Base (RGB)", 2D) = "" {}
    _ContactColor("Color de contacto", Color) = (1.0, 1.0, 0.0)
    _ContactSize("Tam. del contacto", Range(0.01, 100)) = 0.25
}
```

- `_ContactColor`: color utilizado para destacar el contacto.
- `_ContactSize`: distancia máxima a la que el efecto es visible. El `_ContactColor` se va perdiendo hasta que la distancia entre ambos objetos es `_ContactSize`.

El efecto se consigue haciendo que:

- El PS lee del z-buffer la información del objeto en primer plano con
`SAMPLE_DEPTH_TEXTURE(_CameraDepthTexture, sampler_CameraDepthTexture, uv);1`.
- Utilizando la función `LinearEyeDepth(depth, _ZBufferParams)` obtiene la distancia desde la cámara a ese punto (profundidad en el sistema de coordenadas del mundo).
- Con lo anterior calcula la distancia entre el punto que se quiere pintar y el que ya hay.
- La distancia calculada se utiliza con `_ContactSize` para determinar el valor entre 0 y 1 que se utilizará para la interpolación lineal entre el color base del material (textura) y el color máximo del contacto (`_ContactColor`).
- El PS devuelve el valor de la interpolación lineal con todos Los datos calculados previamente.

6. Visible atrás dependiendo de la distancia

El objetivo es hacer un material que en condiciones normales pinte la geometría con una textura y que en caso de ser tapado por algo sea más o menos visible dependiendo de la distancia a la que el objeto se encuentre con respecto al objeto que lo tapa.

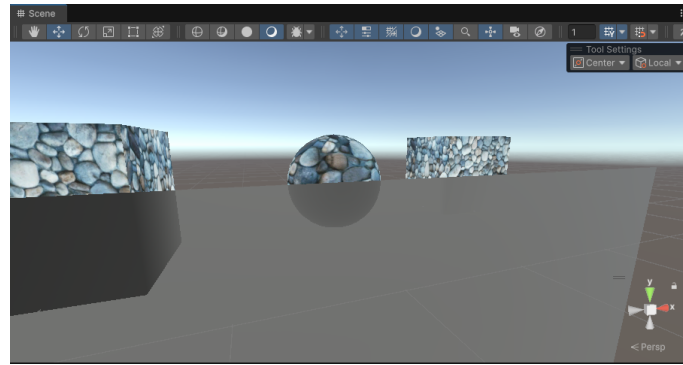
Para probarlo, crea una escena nueva `04VisibleAtrasDistancia`, utiliza el renderer por defecto creado en la sección 2 (`PR06_URenderPipelineAsset_Renderer`) y añade un plano igual que en el efecto anterior.

Crea un shader y material nuevo `04VisibleAtrasDistancia` partiendo también del shader que muestra la textura añadiendo dos parámetros adicionales:

Properties

```
{
    _MainTex ("Base (RGB)", 2D) = "" {}
    _BackgroundColor("Background Color", Color) = (0.0, 0.0, 0.0)
    _Distance("Distancia maxima", Range(0.01, 1000)) = 2
}
```

¹Incluye el fichero `DeclareDepthTexture.hlsl` (en el mismo directorio que `Core.hlsl`) para tener acceso a esa textura.



- `_BackgroundColor`: color utilizado cuando el objeto está pegado con el objeto que lo tapa.
- `_Distance`: distancia máxima a la que el efecto es visible. El `_BackgroundColor` se va perdiendo hasta que la distancia entre el objeto y el objeto tapado es `_Distance`.

La idea del efecto es:

- El color devuelto por el PS tiene el componente alpha de forma que se mezcla con el color ya existente. Si el alpha es 0 se utiliza el color de las etapas previas (`Blend SrcAlpha OneMinusSrcAlpha`).
- El PS calcula la distancia entre el objeto ocluidor y el objeto a pintar de forma parecida a cómo se hacía en el efecto anterior.
- El PS devuelve el color de la textura si el objeto está delante y un valor entre 0 y 1 si está detrás. Será 1 si la distancia es 0 y 0 si la distancia es `_Distance` o mayor.

7. Prueba y reflexión

Este apartado no requiere entregar nada. Incorpora al proyecto el material de color fijo de alguna práctica anterior y crea un material con color gris claro.

Asigna el material en el plano de alguna de las dos últimas prácticas y observa el resultado. ¿Qué explicación le encuentras?

8. Entrega de la práctica

La práctica debe entregarse utilizando el mecanismo de entrega del campus virtual, no más tarde de la fecha y hora indicada en la cabecera de la práctica.

Solo uno de los dos miembros del grupo debe hacer la entrega, subiendo al campus un fichero llamado `GrupoNN.zip` donde NN representa el número de grupo con dos dígitos.

El fichero debe tener exclusivamente:

- El proyecto de Unity con los directorios mínimos necesarios (`Assets`, `Packages` y `ProjectSettings`). El contenido del directorio `Assets` debe ser el que se deduce de las instrucciones anteriores y que puede verse en la figura 1.
- Un fichero `alumnos.txt` donde se indicará el nombre de los componentes del grupo.